

# SmartShield IPS

## Factory Production Deployment Report

**OT/ICS Edition — Semester 2**

Protocols detected: **Modbus • S7Comm • DNP3 • PROFINET • CIP • NTP/PTP**

Features: OT-safe blocking • Redis persistence • Web auth • SIEM syslog export

# 1. System Overview

SmartShield is a Python/Zeek-based Intrusion Prevention System originally built as a graduation project running inside Docker on a laptop. Semester 2 transforms it into a production-ready factory IPS capable of protecting Operational Technology (OT) and Industrial Control System (ICS) environments from real attacks.

The system sits passively on a Linux server at the IT/OT network boundary, receiving a mirrored copy of all traffic via a SPAN port or passive TAP. It detects 20 categories of OT/ICS attacks, generates structured evidence, and optionally blocks attackers through iptables — with a critical OT-safe mechanism that prevents any PLC, HMI, or SCADA server from ever being auto-blocked.

## 1.1 What Changed from Semester 1

| Before (Semester 1) | After (Semester 2)                                                          |
|---------------------|-----------------------------------------------------------------------------|
| Running environment | Docker on laptop → Dedicated Linux server (bare-metal or Docker) at factory |
| Traffic source      | PCAP files / simulated traffic → Live SPAN/TAP port capture                 |
| OT detection        | None → 20 attack types across 6 OT protocols                                |
| Blocking safety     | Blindly blocks any IP → OT-safe: PLCs/HMIs/SCADA servers are protected      |
| Web interface       | No login, unauthenticated API → Flask login with session management         |
| Data persistence    | Redis resets on restart → AOF + RDB persistence, survives reboots           |
| Alert export        | Slack and STIX only → Added Syslog UDP/TCP and HTTP webhook for SIEM        |
| Config              | analysis_direction: out (IT only) → all (detects inbound OT attacks)        |
| Sensitivity         | evidence_detection_threshold 0.25 → 0.15 for faster OT alerts               |

## 1.2 New Files Added

| File                                 | What It Does                                                                                                         |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| modules/ot_detection/ot_detection.py | Main OT module — 20 attack types, 6 OT protocols, sliding-window flood counters, OT-safe blocking, deduplication     |
| zeek-scripts/ot_protocols.zeek       | Zeek script — hooks Modbus PDU events, DNP3 headers, S7Comm connections, PROFINET DCP, PTP, emits structured notices |

| File                                        | What It Does                                                                                     |
|---------------------------------------------|--------------------------------------------------------------------------------------------------|
| modules/exporting_alerts/syslog_exporter.py | Syslog (UDP/TCP, RFC-5424) and HTTP webhook export for SIEM integration                          |
| webinterface/auth.py                        | Flask login page, session guard, 8-hour sessions, credential config via environment variables    |
| config/ot_protected_devices.conf            | List of PLC/HMI/SCADA IPs — these are never auto-blocked                                         |
| config/modbus_authorized_masters.conf       | Authorized Modbus master IPs — others trigger OT_UNAUTHORIZED_MODBUS_ACCESS                      |
| config/redis.conf                           | Production Redis config: AOF persistence, memory limits, dangerous command rename                |
| docker/docker-compose.factory.yml           | Production Docker Compose: Redis volume, healthchecks, restart policies, separated web container |
| install/install_factory.sh                  | One-shot Ubuntu 22.04/24.04 bare-metal installer — Zeek 8, Python, Redis, systemd, credentials   |
| install/smartshield.service                 | Systemd unit file: restart-on-failure, 8-hour sessions, resource limits, boot auto-start         |
| install/environment.template                | Environment variable template for credentials and interface name                                 |

## 2. Factory Network Architecture

SmartShield is deployed at the IT/OT boundary following the ISA-95 / Purdue Model for industrial network segmentation. It monitors all traffic passively — OT devices never know it exists.

### 2.1 Purdue Model Network Zones

| Zone           | Name            | Contains                                                                                         |
|----------------|-----------------|--------------------------------------------------------------------------------------------------|
| Level 0–1      | Field devices   | PLCs, RTUs, VFDs, sensors, actuators. These communicate via Modbus, S7Comm, DNP3, PROFINET, CIP. |
| Level 2        | Control         | HMIIs, engineering workstations, DCS systems. Run TIA Portal, SCADA client software.             |
| Level 3        | Operations      | SCADA servers, historians (OSIsoft PI), OPC-UA servers, MES systems.                             |
| IT/OT Boundary | DMZ             | SmartShield server sits here. SPAN/TAP receives all traffic crossing the boundary.               |
| Level 4–5      | IT / Enterprise | Business systems, ERP, internet. Standard IT security applies above this line.                   |

### 2.2 SmartShield Server Placement

#### Physical Placement Rule

The SmartShield server **MUST** be connected to the network via two separate interfaces:

eth0 (management): SSH access, web UI on port 55000, syslog output. Connected to IT/management VLAN.

eth1 (monitoring): Connected to a SPAN port or TAP at the IT/OT boundary. Receives a mirror of all traffic.

This interface does **NOT** need an IP address.

**NEVER** use your management interface for traffic capture — it will flood the interface and make SSH unusable.

## 2.3 Data Flow Through SmartShield

| Stage               | What Happens                                                                                                  |
|---------------------|---------------------------------------------------------------------------------------------------------------|
| SPAN/TAP port       | Switch mirrors traffic to eth1. SmartShield sees a copy of every packet without disrupting OT communications. |
| Zeek                | Parses raw packets into structured logs: conn.log, modbus.log, dnp3.log, notice.log, etc.                     |
| Redis pub/sub       | Zeek writes events to Redis channels. All SmartShield modules read from Redis, not directly from Zeek.        |
| OT Detection module | Subscribes to new_flow, new_modbus, new_s7comm, new_dnp3, new_notice. Runs detection logic.                   |
| Evidence pipeline   | Evidence objects flow through evidence_handler.py → alert_handler.py → alerts.json                            |
| Blocking module     | Subscribes to new_blocking channel. Checks OT-safe list before adding iptables rules.                         |
| Export module       | Subscribes to export_evidence. Sends to Slack, STIX, Syslog, or HTTP webhook.                                 |
| Web interface       | Flask app reads Redis directly. Shows profiles, alerts, evidence, blocked IPs in real time.                   |

## 3. OT/ICS Attacks Detected

The `modules/ot_detection/ot_detection.py` module detects 20 attack categories. Every detection generates an `Evidence` object that flows through the normal SmartShield pipeline — visible in the web interface, written to `alerts.json`, and exported to any configured SIEM.

### 3.1 Modbus/TCP — Port 502

| Evidence Type                 | Threat Level | Detection Logic                                                                                         |
|-------------------------------|--------------|---------------------------------------------------------------------------------------------------------|
| OT_MODBUS_CONNECTION_FLOOD    | HIGH         | Too many TCP connections/min to port 502 from one source (default threshold: 20/min)                    |
| OT_UNAUTHORIZED_MODBUS_ACCESS | HIGH         | Source IP not in <code>modbus_authorized_masters.conf</code> contacts a protected OT device on port 502 |
| OT_MODBUS_WRITE_COILS         | CRITICAL     | Write Single/Multiple Coils or Registers (func codes 5,6,15,16,22,23) from unauthorized host            |
| OT_MODBUS_ILLEGAL_FUNCTION    | MEDIUM       | Malformed function code (>127) or Modbus exception response from device                                 |

### 3.2 Siemens S7Comm / ISO-TSAP — Port 102

| Evidence Type                | Threat Level | Detection Logic                                                                                   |
|------------------------------|--------------|---------------------------------------------------------------------------------------------------|
| OT_S7COMM_JOB_FLOOD          | HIGH         | High-rate S7 job requests (TCP connections to port 102) within sliding 60s window (threshold: 50) |
| OT_S7COMM_UNAUTHORIZED_READ  | HIGH         | S7Comm connection from a host not in the authorized masters list targeting a protected OT device  |
| OT_S7COMM_UNAUTHORIZED_WRITE | CRITICAL     | S7 WriteVar command (ROSCTR=1, function 0x05) from unauthorized source                            |
| OT_S7COMM_PLC_STOP           | CRITICAL     | S7 STOP/control-plane command (function 0x29 or 0x00, ROSCTR=1) — halts the PLC immediately       |

### 3.3 DNP3 — Port 20000

| Evidence Type                 | Threat Level | Detection Logic                                                                                  |
|-------------------------------|--------------|--------------------------------------------------------------------------------------------------|
| OT_DNP3_FLOOD                 | HIGH         | High-rate DNP3 fragment/request flood (default threshold: 100 requests/60s)                      |
| OT_DNP3_UNAUTHORIZED_FUNCTION | HIGH         | Function code > 33 which is reserved/dangerous in the DNP3 spec — potential exploitation attempt |

### 3.4 Industrial Network Protocols

| Evidence Type           | Threat Level | Detection Logic                                                                                                  |
|-------------------------|--------------|------------------------------------------------------------------------------------------------------------------|
| OT_PROFINET_DCP_FLOOD   | MEDIUM       | PROFINET DCP discovery flood (port 34964 UDP) — can overwhelm embedded PLCs with limited stack memory            |
| OT_CIP_MESSAGE_FLOOD    | HIGH         | EtherNet/IP CIP explicit message flood (port 44818 TCP) — exhausts connection slots on Allen-Bradley controllers |
| OT_NTP_TIME_SYNC_ATTACK | HIGH         | Possible NTP spoofing: > 5 NTP responses in 10s from same source — desynchronizes precision OT clocks            |
| OT_PTP_SPOOFING         | HIGH         | IEEE 1588 PTP event message (port 319 UDP) from a non-OT-known source — grandmaster clock spoofing               |

### 3.5 PLC Command and Process Attacks

| Evidence Type            | Threat Level | Detection Logic                                                                                               |
|--------------------------|--------------|---------------------------------------------------------------------------------------------------------------|
| OT_PLC_MODE_SWITCH       | CRITICAL     | Rapid RUN/STOP mode-switch commands to a PLC (threshold: 5 switches/60s) — can crash the PLC into fault state |
| OT_WATCHDOG_MANIPULATION | CRITICAL     | Watchdog timer disabled or manipulated (from Zeek OT::WatchdogManipulation notice) — safety systems bypassed  |
| OT_VALVE_CYCLING         | CRITICAL     | Repeated open/close valve commands (OT::ValveCycling notice) — causes mechanical wear and process disruption  |
| OT_MOTOR_CYCLING         | CRITICAL     | Rapid motor start/stop commands (OT::MotorCycling notice) — causes overheating and winding damage             |

| Evidence Type       | Threat Level | Detection Logic                                                                                             |
|---------------------|--------------|-------------------------------------------------------------------------------------------------------------|
| OT_PROTOCOL_ANOMALY | MEDIUM       | Generic OT protocol anomaly from Zeek OT notices — catch-all for patterns not covered by specific detectors |

## 3.6 Confidence and Threat Levels

Every Evidence object has a confidence score (0.0–1.0) and a threat level. The `evidence_detection_threshold` in `smartshield.yaml` (set to 0.15 for factory) controls how much accumulated evidence is needed before an Alert is triggered. OT attacks use higher confidence scores (0.85–0.95) because protocol behavior is well-defined.

| Threat Level       | Evidence Types Assigned                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CRITICAL (0.95)    | OT_S7COMM_PLC_STOP, OT_MODBUS_WRITE_COILS, OT_PLC_MODE_SWITCH, OT_WATCHDOG_MANIPULATION, OT_VALVE_CYCLING, OT_MOTOR_CYCLING                                                       |
| HIGH (0.80–0.90)   | OT_MODBUS_CONNECTION_FLOOD, OT_UNAUTHORIZED_MODBUS_ACCESS, OT_S7COMM_JOB_FLOOD, OT_DNP3_FLOOD, OT_CIP_MESSAGE_FLOOD, OT_NTP_TIME_SYNC_ATTACK, OT_PTP_SPOOFING, OT_PLC_MODE_SWITCH |
| MEDIUM (0.70–0.80) | OT_MODBUS_ILLEGAL_FUNCTION, OT_DNP3_UNAUTHORIZED_FUNCTION, OT_PROFINET_DCP_FLOOD, OT_PROTOCOL_ANOMALY                                                                             |



## 4. OT-Safe Blocking

If SmartShield ever blocks a PLC's IP address with iptables, that PLC stops receiving commands from the SCADA system.

The factory production line halts. This is the single most dangerous operational risk of running an IPS in an OT environment.

The OT-safe blocking system prevents this. PLCs, HMIs, and SCADA servers listed in `config/ot_protected_devices.conf` are NEVER passed to iptables — no matter what evidence is detected against them.

### 4.1 How OT-Safe Blocking Works

The blocking pipeline has been modified at three levels:

1. `modules/blocking/blocking.py` — `_is_ot_protected()` is called before every iptables rule. If the IP is protected, the rule is skipped and a `[OT-SAFE]` entry is written to `blocking.log`.
2. `smartshield_files/core/database/redis_db/alert_handler.py` — New `mark_as_ot_protected()`, `is_ot_protected()`, `get_ot_protected_ips()` methods persist the protected set in Redis key `ot_protected_ips`.
3. `modules/ot_detection/ot_detection.py` — When evidence is set against a protected OT IP, the description includes `[OT-SAFE: blocking suppressed]` so analysts know immediately.

**What is NOT blocked:** The protected OT device's own IP address (e.g. the PLC at 192.168.10.10).

**What IS still blocked:** External IT-side hosts attacking OT devices. If 10.5.1.88 is attacking PLC 192.168.10.10 with a Modbus flood, the attacker IP 10.5.1.88 is blocked normally — only the PLC itself is protected.

**What always happens:** Evidence, alerts, and SIEM exports are generated regardless of blocking status.

### 4.2 Configuring OT Protected Devices

Edit `config/ot_protected_devices.conf`. Add one IP address or CIDR range per line. Lines starting with `#` are comments.

```
# config/ot_protected_devices.conf
#
# Level 0-1: PLCs, RTUs, VFDs (field devices)
192.168.10.10 # PLC-01 Siemens S7-1200 (Assembly Line A)
192.168.10.11 # PLC-02 Siemens S7-1500 (Assembly Line B)
192.168.10.12 # PLC-03 Allen-Bradley (Conveyor system)
192.168.10.20 # RTU-01 Remote Terminal Unit (substation)
192.168.10.30 # VFD-01 Variable Frequency Drive

# Level 2: HMIs and Engineering Workstations
192.168.20.10 # HMI-01 SCADA operator workstation
192.168.20.11 # HMI-02 Control room display
```

```
192.168.20.50 # EWS-01 Engineering workstation (TIA Portal)

# Level 3: SCADA Servers and Historians
192.168.30.10 # SCADA-Server-01
192.168.30.20 # Historian-01 OSIsoft PI
192.168.30.30 # OPC-UA Server

# CIDR ranges are also supported:
# 192.168.10.0/24 # Protect entire field device subnet
```

## 4.3 Configuring Authorized Modbus Masters

Edit `config/modbus_authorized_masters.conf`. List every IP that is legitimately allowed to send Modbus read/write commands. Any unlisted host contacting port 502 on a protected OT device triggers `OT_UNAUTHORIZED_MODBUS_ACCESS`.

```
# config/modbus_authorized_masters.conf
# One exact IP per line. CIDR not supported here.
192.168.20.10 # HMI-01 (authorized to read/write PLCs)
192.168.20.50 # EWS-01 (engineering workstation, TIA Portal)
192.168.30.10 # SCADA-Server-01 (authorized polling)
```

### Leave both files empty initially

If you leave `ot_protected_devices.conf` empty, OT-safe blocking is still active but no IPs are protected.

If you leave `modbus_authorized_masters.conf` empty, unauthorized-access detection is disabled (flood detection still works).

Populate both files before enabling blocking in production.

## 5. Installation

### 5.1 Server Requirements

| Requirement          | Specification                                                                                 |
|----------------------|-----------------------------------------------------------------------------------------------|
| Operating system     | Ubuntu 22.04 LTS or 24.04 LTS (bare-metal or VM). Debian 12 also supported.                   |
| CPU                  | 4+ cores recommended. Zeek is multi-threaded and can use 2–3 cores under load.                |
| RAM                  | 8 GB minimum. 16 GB recommended for busy factory networks (> 1 Gbps sustained).               |
| Disk                 | 50 GB minimum for OS, SmartShield, and Zeek logs. 200 GB recommended if storing rotated logs. |
| Network interfaces   | Minimum 2: one management (eth0), one monitoring (eth1) connected to SPAN/TAP.                |
| Monitoring interface | Does not need an IP address. Can be any speed as long as it matches the SPAN output.          |
| Python               | Python 3.10+ (included in Ubuntu 22.04/24.04).                                                |
| Zeek                 | Zeek 8.0 — installed automatically by install_factory.sh.                                     |
| Redis                | Redis 7 — installed and configured automatically.                                             |
| Root access          | Required for installation, iptables, and packet capture.                                      |

### 5.2 Pre-Installation: Switch SPAN Port Setup

Before installing SmartShield, configure your managed switch to mirror the IT/OT boundary port to the server's monitoring interface. This is done in the switch management interface — the exact steps depend on the switch vendor.

| Switch Vendor        | Source Command                                                                  | Destination/Notes                                                     |
|----------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Cisco IOS / Catalyst | monitor session 1 source interface Gi1/0/1                                      | monitor session 1 destination interface Gi1/0/24                      |
| Juniper EX           | set ethernet-switching-options analyzer SPAN input ingress interface ge-0/0/1.0 | set analyzer SPAN output interface ge-0/0/24.0                        |
| HP / Aruba ProCurve  | mirror-session 1 port A1 mirror-port A24                                        | (via web UI: Diagnostics → Mirror)                                    |
| Siemens SCALANCE     | Port Mirroring under Diagnostics menu                                           | Source: IT/OT boundary port, Destination: SmartShield monitoring port |

| Switch Vendor   | Source Command                                               | Destination/Notes                                                 |
|-----------------|--------------------------------------------------------------|-------------------------------------------------------------------|
| Generic / other | Look for: Port Mirroring, SPAN, Port Monitor, Traffic Mirror | Source = IT/OT boundary uplink,<br>Destination = SmartShield eth1 |

### i Verify SPAN is working before installing SmartShield

After configuring the SPAN port, SSH into the future SmartShield server and run:

```
sudo ip link set eth1 up
sudo tcpdump -i eth1 -n -c 50
```

You should see traffic from your OT devices. If the interface is silent, the SPAN port is not configured correctly.

## 5.3 Option A: Bare-Metal Install (Recommended for Production)

The included install script handles everything on a fresh Ubuntu server. Run it as root:

```
# Step 1 — Copy the project to the factory server
scp -r SmartShield-Factory/ admin@<server-ip>:/opt/

# Step 2 — SSH into the server
ssh admin@<server-ip>

# Step 3 — Run the installer as root
cd /opt/SmartShield-Factory
sudo chmod +x install/install_factory.sh
sudo ./install/install_factory.sh

# The script will:
# - Install Zeek 8, Python 3, Redis, iptables-persistent
# - Ask for the monitoring interface name (e.g. eth1)
# - Ask for web UI username and password (min 12 chars)
# - Copy config to /etc/smartshield/
# - Create /var/lib/smartshield/ for persistent Redis data
# - Install and enable the smartshield systemd service
# - Optionally start SmartShield immediately
```

### After installation — edit OT config files:

```
# Add your factory's PLC/HMI/SCADA IPs (CRITICAL — do before starting)
sudo nano /opt/SmartShield-Factory/config/ot_protected_devices.conf

# Add authorized Modbus master IPs
sudo nano /opt/SmartShield-Factory/config/modbus_authorized_masters.conf

# Start the service
sudo systemctl start smartshield
sudo systemctl status smartshield
```

## 5.4 Option B: Docker (Lab / Pre-Production Testing)

Use Docker if you want to test before deploying bare-metal, or if you are running in a virtualized environment.

```
# Step 1 - Create the .env file
cat > docker/.env << 'ENVEOF'
SMARTSHIELD_INTERFACE=eth1
SMARTSHIELD_WEB_USER=admin
SMARTSHIELD_WEB_PASSWORD=YourStrongFactoryPassword!
SMARTSHIELD_SECRET_KEY=$(python3 -c "import secrets; print(secrets.token_hex(32))")
ENVEOF

# Step 2 - Create persistent storage directories on the host
sudo mkdir -p /var/lib/smartshield/{redis,output}

# Step 3 - Build the Docker image
docker build -f docker/Dockerfile -t smartshield_gp:latest .

# Step 4 - Start all services
docker compose -f docker/docker-compose.factory.yml up -d

# Step 5 - Check status
docker compose -f docker/docker-compose.factory.yml ps
docker logs smartshield -f
```

### Docker vs Bare-Metal

Docker is easier to set up but has limitations in a factory:

- iptables blocking requires `network_mode: host` which reduces container isolation
- Live interface capture with `NET_RAW` needs the host interface to be UP before starting
- Redis volumes must be bind-mounted to a host path (`/var/lib/smartshield/redis`) for true persistence

Bare-metal is recommended for production deployments where stability is critical.

## 6. Configuration Reference

### 6.1 Main Configuration: config/smartshield.yaml

The key parameters changed for factory deployment (from the original defaults):

```
parameters:
  # CHANGED: detect inbound attacks FROM IT side TO OT devices
  analysis_direction: all    # was: out

  # CHANGED: keep evidence history across server reboots
  deletePrevdb: false       # was: true

  # CHANGED: rotate and keep Zeek logs for 1 week
  rotation: true
  rotation_period: 1day
  keep_rotated_files_for: 7 day

detection:
  # CHANGED: lower threshold so OT attacks alert faster
  evidence_detection_threshold: 0.15  # was: 0.25

ot_detection:
  ot_devices_file: config/ot_protected_devices.conf
  modbus_masters_file: config/modbus_authorized_masters.conf
  modbus_flood_threshold: 20          # connections/minute before alerting
  s7_job_flood_threshold: 50          # S7 connections/minute
  dnp3_flood_threshold: 100           # DNP3 requests/minute
  profinet_dcp_threshold: 50         # PROFINET DCP requests/minute
  cip_flood_threshold: 100            # CIP messages/minute
  plc_mode_switch_threshold: 5        # RUN/STOP switches in 60s
  flood_window_seconds: 60            # sliding window duration

web_interface:
  port: 55000
```

### 6.2 SIEM / Syslog Export Configuration

Add `syslog` and/or `webhook` to the `export_to` list in the `exporting_alerts` section:

```
exporting_alerts:
  export_to: [syslog]          # Options: slack, stix, syslog, webhook

  # Syslog target — Splunk, QRadar, Wazuh, Graylog, Elastic SIEM
  syslog_server: 192.168.1.100 # Your SIEM server IP
  syslog_port: 514             # 514 for UDP, 601 for TCP syslog
  syslog_protocol: udp         # "udp" or "tcp"
  syslog_facility: local0      # local0 through local7, or daemon, user
  syslog_app_name: smartshield

  # HTTP Webhook — Splunk HEC, Grafana, PagerDuty, custom endpoint
  webhook_url: http://siem-server:8088/services/collector
  webhook_auth_token: your_splunk_hec_token
  webhook_timeout: 5           # seconds before giving up
```

The syslog message format is structured for easy SIEM parsing:

```
smartshield: SmartShield ALERT | type=OT_MODBUS_WRITE_COILS |
  threat=CRITICAL | confidence=0.95 | attacker=10.5.1.88 |
```

```
ts=2024/03/15 14:32:01.000000+0000 |
desc=Unauthorized Modbus write from 10.5.1.88 to OT device 192.168.10.10...
```

## 6.3 Web Interface Credentials

Set credentials via environment variables — never hardcode them in the config file:

```
# For bare-metal: edit /etc/smartshield/environment
SMARTSHIELD_WEB_USER=factoryadmin
SMARTSHIELD_WEB_PASSWORD=MyFactory_Secure_Pass2024!

# Generate a cryptographic secret key (run this once):
python3 -c "import secrets; print(secrets.token_hex(32))"
SMARTSHIELD_SECRET_KEY=<paste output here>

# For Docker: set in docker/.env file
# Restart after changes: sudo systemctl restart smartshield
```

## 6.4 Redis Persistence Configuration

The new `config/redis.conf` enables both AOF (append-only file) and RDB snapshots:

```
# AOF persistence — writes every command to disk
appendonly yes
appendfilename "smartshield.aof"
appendfsync everysec      # flush to disk every second (best balance)

# RDB snapshots — periodic full dump as backup
save 900 1                # save after 900s if at least 1 key changed
save 300 10               # save after 300s if at least 10 keys changed
save 60 10000             # save after 60s if at least 10000 keys changed

# Memory limit — adjust based on available RAM
maxmemory 512mb
maxmemory-policy allkeys-lru

# Dangerous commands disabled in production
rename-command FLUSHALL ""
rename-command FLUSHDB ""
```

Data is stored in `/var/lib/smartshield/redis/` on the host. This directory persists across reboots and Docker container restarts.

## 7. Running and Managing SmartShield

### 7.1 Starting for the First Time — Recommended Sequence

4. **Populate OT config files.** Add all PLC/HMI/SCADA IPs to `ot_protected_devices.conf` and all authorized Modbus master IPs to `modbus_authorized_masters.conf`. This is mandatory before enabling blocking.
5. **Start in monitor-only mode first.** Do not enable `--blocking` on the first run. Let SmartShield observe for 1–2 weeks to establish a normal baseline. Watch for false positives.
6. **Verify SPAN capture.** Run `sudo tcpdump -i eth1 -n port 502 or port 102 -c 20` to confirm OT traffic is arriving on the monitoring interface.
7. **Check the web interface.** Open `http://<server-ip>:55000` and log in. You should see profiles appearing within a few minutes of capture starting.
8. **Tune thresholds.** Review alerts after 1 week. Adjust `modbus_flood_threshold` and `s7_job_flood_threshold` in `smartshield.yaml` based on your factory's normal polling rates.
9. **Enable blocking.** Add `--blocking` to the start command in the `systemd` service or `docker-compose`, then restart. Verify no OT device IPs appear in the `iptables` rules: `sudo iptables -L smartshieldBlocking -n`
10. **Configure SIEM export.** Add `syslog` to `export_to` and set `syslog_server`, then restart. Verify alerts arrive in your SIEM.

### 7.2 Systemd Commands (Bare-Metal)

| Command                                                          | Description                                                 |
|------------------------------------------------------------------|-------------------------------------------------------------|
| <code>sudo systemctl status smartshield</code>                   | Check if running. Shows last 20 log lines.                  |
| <code>sudo systemctl start smartshield</code>                    | Start the service.                                          |
| <code>sudo systemctl stop smartshield</code>                     | Stop the service. Redis data is preserved.                  |
| <code>sudo systemctl restart smartshield</code>                  | Restart after config changes.                               |
| <code>sudo systemctl enable smartshield</code>                   | Enable auto-start on boot (already done by install script). |
| <code>sudo journalctl -u smartshield -f</code>                   | Live log stream (Ctrl+C to stop).                           |
| <code>sudo journalctl -u smartshield --since "1h ago"</code>     | Logs from last hour.                                        |
| <code>sudo journalctl -u smartshield --since "2024-03-15"</code> | Logs from a specific date.                                  |
| <code>sudo journalctl -u smartshield -p err</code>               | Show only errors.                                           |



## 7.3 Docker Commands

| Command                                                                   | Description                                            |
|---------------------------------------------------------------------------|--------------------------------------------------------|
| <code>docker compose -f docker/docker-compose.factory.yml up -d</code>    | Start all services in background.                      |
| <code>docker compose -f docker/docker-compose.factory.yml down</code>     | Stop all services. Volumes (Redis data) are preserved. |
| <code>docker compose -f docker/docker-compose.factory.yml restart</code>  | Restart all services.                                  |
| <code>docker compose -f docker/docker-compose.factory.yml logs -f</code>  | Follow all container logs.                             |
| <code>docker logs smartshield -f --tail 100</code>                        | Follow SmartShield container logs only.                |
| <code>docker logs smartshield-redis -f</code>                             | Follow Redis container logs.                           |
| <code>docker stats</code>                                                 | Live CPU and memory usage for all containers.          |
| <code>docker exec -it smartshield-redis redis-cli ping</code>             | Test Redis connectivity.                               |
| <code>docker exec -it smartshield-redis redis-cli info persistence</code> | Check AOF/RDB persistence status.                      |

## 7.4 Useful Diagnostic Commands

```
# — Network verification —————
# Check that OT traffic is arriving on the monitoring interface
sudo tcpdump -i eth1 -n port 502 or port 102 or port 20000 -c 30

# Count Modbus packets in last 10 seconds
sudo timeout 10 tcpdump -i eth1 -n port 502 | wc -l

# — iptables blocking rules —————
sudo iptables -L smartshieldBlocking -v -n

# Verify no OT device IPs are in the blocked list
sudo iptables -L smartshieldBlocking -n | grep 192.168.10

# — Redis status —————
# Check persistence status
redis-cli INFO persistence

# Check OT-protected IPs loaded into Redis
redis-cli SMEMBERS ot_protected_ips

# Count total evidence stored
redis-cli KEYS "evidence_*" | wc -l

# Count alerts triggered
redis-cli KEYS "alert_*" | wc -l

# — SmartShield output —————
ls -lht /var/lib/smartshield/output/
tail -f /var/lib/smartshield/output/alerts.json
cat /var/lib/smartshield/output/blocking.log

# — Zeek status —————
```

```
zeek --version  
ls /opt/zeek/logs/current/
```

## 8. Web Interface

### 8.1 Accessing the Web Interface

| Setting     | Value                                                                     |
|-------------|---------------------------------------------------------------------------|
| URL         | http://<server-management-ip>:55000                                       |
| Username    | Set in SMARTSHIELD_WEB_USER environment variable (default: admin)         |
| Password    | Set in SMARTSHIELD_WEB_PASSWORD (default shows warning — must be changed) |
| Session     | 8-hour sessions. Expires automatically. All routes require login.         |
| Port change | Edit web_interface.port in smartshield.yaml, then restart.                |

#### Restrict access to port 55000

The web interface should only be accessible from your factory management VLAN.

Add a firewall rule to restrict access:

```
sudo ufw allow from 192.168.20.0/24 to any port 55000
sudo ufw deny 55000
```

Never expose port 55000 to the internet. Use a VPN if remote access is needed.

### 8.2 Web Interface Pages

| Page                           | Contents                                                                     |
|--------------------------------|------------------------------------------------------------------------------|
| Overview (/)                   | Summary of profiles, alert count, analysis start time, input type.           |
| Analysis (/analysis)           | Per-IP profiles, time windows, evidence list, flow details, blocking status. |
| Timeline                       | Chronological event timeline for each IP — shows attack progression.         |
| Documentation (/documentation) | Built-in SmartShield documentation reference.                                |

### 8.3 Authentication Implementation

The `webinterface/auth.py` module adds login protection using Flask sessions. Key design decisions:

- **No external dependencies:** Uses Flask's built-in session system with a secret key. No database required for auth.
- **Before-request guard:** Every request (except `/login`, `/logout`, `/static`) is intercepted and redirected to login if no valid session exists.

- **Credential source:** Usernames and passwords come from environment variables only — not from config files that might be committed to git.
- **Session lifetime:** 8 hours. Configured via `app.permanent_session_lifetime`. Survives page reloads but expires on timeout.
- **Brute-force protection:** Failed login attempts add a 0.5-second delay to slow automated attacks.
- **Secret key:** Must be a random 32-byte hex string. If not set via `SMARTSHIELD_SECRET_KEY`, a random key is generated on each restart (invalidating all existing sessions).

## 9. OT Detection Module — Technical Details

### 9.1 Module Architecture

The OT Detection module (`modules/ot_detection/ot_detection.py`) follows the same IModule interface as all other SmartShield modules. It is automatically discovered by the process manager because the file name matches the directory name (`ot_detection/ot_detection.py`).

It subscribes to five Redis pub/sub channels:

| Channel                 | Source and Contents                                                                                                                   |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>new_flow</code>   | Every Zeek conn.log entry — used for flood detection on OT ports (502, 102, 20000, 34964, 44818, 123, 319)                            |
| <code>new_notice</code> | Zeek notice.log entries — catches PLCModeSwitch, WatchdogManipulation, ValveCycling, MotorCycling from <code>ot_protocols.zeek</code> |
| <code>new_modbus</code> | Per-PDU Modbus events from Zeek OT parser — function code, exception flag, coil values                                                |
| <code>new_s7comm</code> | Per-PDU S7Comm events — ROSCTR, function code, read/write operations                                                                  |
| <code>new_dnp3</code>   | Per-PDU DNP3 events — function code, <code>is_unsolicited</code> , data objects                                                       |

### 9.2 Sliding Window Flood Detection

All flood counters use sliding time windows (not fixed intervals) to avoid boundary-crossing attacks. The implementation:

```
# Example: Modbus flood detection sliding window
now = time.time()

# Append new connection timestamp
self._modbus_conns[saddr].append(now)

# Evict timestamps older than flood_window_seconds
self._modbus_conns[saddr] = [
    t for t in self._modbus_conns[saddr]
    if now - t <= self._flood_window # default: 60 seconds
]

# Count connections in the current window
count = len(self._modbus_conns[saddr])

# Trigger evidence if threshold exceeded
if count >= self._modbus_flood_threshold: # default: 20
    self._set_evidence(ev_type=EvidenceType.OT_MODBUS_CONNECTION_FLOOD, ...)
```

### 9.3 Alert Deduplication

The module maintains a `self._alerted` set of alert keys (format: `"modbus_flood:<srcip>:<twid>"`) to prevent the same attack from generating hundreds of

duplicate evidence objects. The set is per-instance (resets when the module restarts). Within a time window, each unique (attack\_type, source\_ip, dest\_ip) combination generates exactly one Evidence object.

## 9.4 Zeek OT Protocol Script

The `zeek-scripts/ot_protocols.zeek` script is loaded via `zeek-scripts/__load__.zeek`. It:

- Hooks into Zeek's built-in Modbus policy (`modbus_message`, `modbus_write_single_coil_request`, `modbus_exception`, etc.)
- Hooks into Zeek's built-in DNP3 policy (`dnp3_application_request_header`)
- Tracks S7Comm connections at the TCP level (port 102) since Zeek has no full S7 parser
- Detects PROFINET DCP floods on port 34964 UDP via `udp_request` events
- Detects PTP (IEEE 1588) traffic on port 319 UDP
- Defines custom Notice types: `PLCModeSwitch`, `WatchdogManipulation`, `ValveCycling`, `MotorCycling`, `ModbusWriteCoil`, `ModbusIllegalFunction`, `S7CommPLCStop`, `DNP3UnauthorizedFunc`

If Zeek's OT policy bundle is not available (older Zeek installations), `ot_protocols.zeek` will partially fail but SmartShield falls back gracefully to `conn.log`-based detection which still handles flood attacks on OT ports.

## 9.5 Evidence Object Structure

Every OT detection creates a fully structured Evidence object compatible with the SmartShield evidence pipeline:

```
Evidence(  
  evidence_type = EvidenceType.OT_MODBUS_WRITE_COILS,  
  description   = "Unauthorized Modbus write from 10.5.1.88 to OT device",  
  attacker     = Attacker(direction=Direction.SRC, ioc_type=IoCType.IP,  
                           value="10.5.1.88", profile=ProfileID(ip="10.5.1.88")),  
  victim       = Victim(direction=Direction.DST, ioc_type=IoCType.IP,  
                           value="192.168.10.10"),  
  threat_level = ThreatLevel.CRITICAL,  
  profile      = ProfileID(ip="10.5.1.88"),  
  timewindow   = TimeWindow(number=3),  
  uid          = ["CYbGfv3fLZzYMQj12"],  
  timestamp    = "2024/03/15 14:32:01.000000+0000",  
  proto        = Proto.TCP,  
  dst_port     = 502,  
  confidence   = 0.95,  
)
```

## 10. Tuning and Optimization

### 10.1 Baseline Period

OT networks have highly predictable, cyclic traffic patterns — PLCs poll sensors at fixed intervals, HMIs refresh at fixed rates. This predictability is a strength for detection but requires careful tuning to avoid false positives.

#### Recommended tuning approach

Week 1–2: Monitor only (no blocking). Collect baseline data. Note the normal Modbus polling rate from your SCADA server.

Week 3: Lower flood thresholds toward the observed normal + 30% headroom. Watch for false positives.

Week 4: Enable blocking with OT-safe list fully populated. Start with high thresholds and tighten over time.

Ongoing: Review blocking.log weekly. Any [OT-SAFE] entries indicate attacks on OT devices that need manual investigation.

### 10.2 Flood Threshold Tuning Guide

| Parameter                 | How to Tune                         | Example                                                                                           |
|---------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------|
| modbus_flood_threshold    | Your SCADA polling rate * 2         | If SCADA polls 5 PLCs every 5s<br>→ normal rate = 60<br>connections/min → set threshold<br>to 120 |
| s7_job_flood_threshold    | TIA Portal polling rate * 3         | Engineering workstations poll<br>slowly (~1 req/sec). Attacks are<br>50+/sec. Keep at 50.         |
| dnp3_flood_threshold      | RTU polling rate * 3                | RTUs typically send 1–5<br>frames/second. Attacks send<br>100+/sec. Start at 100.                 |
| profinet_dcp_threshold    | Near 0 in normal operation          | DCP discovery is only sent<br>during device startup. Any<br>sustained rate > 10 is<br>suspicious. |
| plc_mode_switch_threshold | Set to 2 if one SCADA can<br>switch | A legitimate mode switch is 1 per<br>maintenance window. 5 in 60s is<br>always an attack.         |

## 10.3 Evidence Detection Threshold

The global `evidence_detection_threshold` (in `detection:` section of `smartshield.yaml`) controls how much accumulated threat level is needed before an Alert is generated:

| Threshold | Effect                                                                                                      |
|-----------|-------------------------------------------------------------------------------------------------------------|
| 0.08      | Very sensitive. Set this during initial testing. Expect false positives. Good for validating detections.    |
| 0.15      | Current factory setting. Balanced for OT environments where attacks are often sudden and high-confidence.   |
| 0.25      | Original default. Suitable for IT networks with noisy traffic. Too slow for OT attack response.             |
| 0.43      | Insensitive. Only generates alerts after many corroborating evidence objects. Risk of missing fast attacks. |

## 10.4 Performance Considerations

- Zeek CPU: Zeek is the heaviest process. At 1 Gbps sustained, it uses 2–3 cores. Scale server cores accordingly.
- Redis memory: Default limit is 512 MB in the factory `redis.conf`. Each IP profile uses ~50 KB. 512 MB supports ~10,000 concurrent IP profiles.
- OT detection overhead: The OT module adds negligible CPU. Flood detection uses in-memory lists with  $O(n)$  cleanup —  $n$  is bounded by the threshold value.
- Log rotation: `rotation: true` and `rotation_period: 1day` is set in `smartshield.yaml`. Zeek log files are deleted after `keep_rotated_files_for: 7` day to prevent disk exhaustion.
- Web interface: If the web interface is slow, it is usually Redis query latency. Check `redis-cli INFO` stats for `slowlog`.



# 11. Troubleshooting

## 11.1 Common Problems and Solutions

| Problem                                | First Check                                                                         | Solution/Cause                                                                                                                                                              |
|----------------------------------------|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| OT module not loading                  | Check: <code>journalctl -u smartshield   grep OT</code>                             | Cause: Missing <code>__init__.py</code> . Fix: touch <code>modules/ot_detection/__init__.py</code> and restart.                                                             |
| No Modbus alerts despite flood traffic | Check: <code>tcpdump -i eth1 port 502</code>                                        | Cause 1: SPAN port not configured. Cause 2: <code>modbus_flood_threshold</code> too high. Cause 3: <code>ot_protected_devices.conf</code> has the attacker IP.              |
| Web UI shows no alerts after 1 hour    | Check: <code>redis-cli KEYS "alert_*</code>                                         | Cause: <code>evidence_detection_threshold</code> too high, or <code>time_window_width</code> is 3600s (alerts only appear after 1 TW). Lower threshold to 0.08 for testing. |
| Redis data lost after restart          | Check: <code>redis-cli INFO persistence</code>                                      | Look for <code>aof_enabled:1</code> . Fix: Verify <code>redis.conf</code> has <code>appendonly yes</code> and path is writable.                                             |
| Cannot log into web interface          | Try: <code>curl -l http://localhost:55000/login</code>                              | Cause: <code>SMARTSHIELD_WEB_USER</code> or <code>SMARTSHIELD_WEB_PASSWORD</code> not set. Check environment file and restart service.                                      |
| Syslog not reaching SIEM               | Test: <code>echo "test"   nc -u &lt;siem-ip&gt; 514</code>                          | Check firewall between SmartShield and SIEM. Verify <code>syslog_server</code> and <code>syslog_port</code> in <code>yaml</code> .                                          |
| Factory line halted — PLC was blocked  | URGENT: <code>sudo iptables -D smartshieldBlocking -s &lt;plc-ip&gt; -j DROP</code> | Then immediately add PLC IP to <code>ot_protected_devices.conf</code> and restart SmartShield.                                                                              |
| Zeek OT script load error              | Check: <code>zeek -i eth1 zeek-scripts/ 2&gt;&amp;1   head -30</code>               | Cause: OT policy bundle not installed. <code>zkg install zeek/DINA-community/ICS-Passive-Discovery</code> or ignore — <code>conn.log</code> detection still works.          |
| High CPU usage                         | Check: <code>top -p \$(pgrep -f smartshield.py)</code>                              | Cause: Very high traffic rate. Enable <code>cpu_profiler_enable: true</code> in <code>smartshield.yaml</code> to identify bottleneck.                                       |

| Problem                                       | First Check                          | Solution/Cause                                                                                                 |
|-----------------------------------------------|--------------------------------------|----------------------------------------------------------------------------------------------------------------|
| OT_UNAUTHORIZED_MODBUS_ACCESS false positives | Check modbus_authorized_masters.conf | Add the source IP if it is a legitimate Modbus master. Or leave the file empty to disable this specific check. |

## 11.2 Emergency Procedures

### If SmartShield blocks an OT device (factory emergency)

Step 1 — Immediately remove the iptables rule:

```
sudo iptables -D smartshieldBlocking -s <plc-ip> -j DROP
sudo iptables -D smartshieldBlocking -d <plc-ip> -j DROP
```

Step 2 — Add the PLC IP to ot\_protected\_devices.conf:

```
echo "192.168.10.10 # PLC-01 EMERGENCY ADDED" >> config/ot_protected_devices.conf
```

Step 3 — Restart SmartShield to reload the protected list:

```
sudo systemctl restart smartshield
```

Step 4 — Verify the IP no longer appears in blocking rules:

```
sudo iptables -L smartshieldBlocking -n | grep 192.168.10.10
```

Step 5 — Investigate the original alert to determine if there was a real attack.

## 11.3 Log File Locations

| Log File        | Location and Contents                                                                         |
|-----------------|-----------------------------------------------------------------------------------------------|
| alerts.json     | /var/lib/smartshield/output/alerts.json — All generated alerts in JSON format                 |
| blocking.log    | /var/lib/smartshield/output/blocking.log — All blocking/unblocking events with [OT-SAFE] tags |
| smartshield.log | /var/lib/smartshield/output/smartshield.log — Main application log                            |
| errors.log      | /var/lib/smartshield/output/errors.log — Error and exception log                              |
| Zeek conn.log   | /var/lib/smartshield/output/zeek/conn.log — All network connections parsed by Zeek            |
| Redis AOF       | /var/lib/smartshield/redis/smartshield.aof — Persistent Redis command log                     |
| systemd journal | journalctl -u smartshield — Full service log including startup/shutdown                       |

## 12. Security Hardening Checklist

---

Before going live in a factory environment, verify each item:

### Network Hardening

- **SPAN port:** Monitoring interface (eth1) has no IP address assigned. Run: `ip addr show eth1` — should show no inet line.
- **Firewall:** Port 55000 is only accessible from management VLAN. Verify: `sudo ufw status numbered`
- **Management interface:** eth0 has only necessary ports open (22/SSH, 55000/Web). No services listening on eth1.
- **Redis binding:** Redis only listens on 127.0.0.1 (confirmed in `redis.conf`: `bind 127.0.0.1`)

### Authentication Hardening

- **Default password changed:** `SMARTSHIELD_WEB_PASSWORD` is not `SmartShield2024!`. Verify no warning banner in web UI.
- **Secret key set:** `SMARTSHIELD_SECRET_KEY` is a 32-byte random hex string. If not set, sessions invalidate on restart.
- **Environment file permissions:** `chmod 600 /etc/smartshield/environment` — only root can read credentials.
- **No credentials in git:** Verify `.env` and environment files are in `.gitignore`

### OT Safety Verification

- **OT device list populated:** `config/ot_protected_devices.conf` has all PLCs, HMIs, and SCADA servers.
- **Protected IPs in Redis:** `redis-cli SMEMBERS ot_protected_ips` — should list all configured OT IPs.
- **No OT IPs blocked:** `sudo iptables -L smartshieldBlocking -n` — must not show any PLC/HMI/SCADA IPs.
- **blocking.log reviewed:** No unexpected [OT-SAFE] entries — each one means an attack hit a protected OT device.

### Data Persistence Verification

- **AOF enabled:** `redis-cli INFO persistence` — shows `aof_enabled:1` and `aof_rewrite_in_progress:0`.
- **AOF file exists:** `ls -lh /var/lib/smartshield/redis/smartshield.aof` — should be non-empty after 5 minutes.
- **Reboot test:** `sudo reboot`, then verify SmartShield auto-starts and evidence history is preserved.

### Monitoring and Alerting

- **Syslog tested:** Trigger a test alert and verify it arrives in SIEM within 30 seconds.

- **Systemd watchdog:** Restart policy is on-failure. Test: `sudo kill -9 $(pgrep -f smartshield.py)` — service should restart within 10 seconds.
- **Disk space monitor:** Set up a cron job or monitoring alert if `/var/lib/smartshield` exceeds 80% full.
- **Health check script:** Consider a cron job that pings `http://localhost:55000` and sends alert if web UI is down.

# 13. Quick Reference

## 13.1 All OT Evidence Types

| Evidence Type                 | Threat Level | Description                                         |
|-------------------------------|--------------|-----------------------------------------------------|
| OT_MODBUS_CONNECTION_FLOOD    | HIGH         | Port 502 TCP flood — too many connections/min       |
| OT_UNAUTHORIZED_MODBUS_ACCESS | HIGH         | Modbus from non-authorized master IP                |
| OT_MODBUS_WRITE_COILS         | CRITICAL     | Unauthorized Modbus write coil/register command     |
| OT_MODBUS_ILLEGAL_FUNCTION    | MEDIUM       | Malformed/exception Modbus function code            |
| OT_S7COMM_JOB_FLOOD           | HIGH         | S7 port 102 connection flood                        |
| OT_S7COMM_UNAUTHORIZED_READ   | HIGH         | S7 connection from non-authorized source            |
| OT_S7COMM_UNAUTHORIZED_WRITE  | CRITICAL     | S7 WriteVar from unauthorized host                  |
| OT_S7COMM_PLC_STOP            | CRITICAL     | S7 STOP command sent to PLC                         |
| OT_DNP3_FLOOD                 | HIGH         | DNP3 request flood (port 20000)                     |
| OT_DNP3_UNAUTHORIZED_FUNCTION | HIGH         | DNP3 function code > 33                             |
| OT_PROFINET_DCP_FLOOD         | MEDIUM       | PROFINET DCP discovery flood (port 34964)           |
| OT_CIP_MESSAGE_FLOOD          | HIGH         | EtherNet/IP CIP explicit message flood (port 44818) |
| OT_NTP_TIME_SYNC_ATTACK       | HIGH         | NTP spoofing — > 5 NTP responses/10s                |
| OT_PTP_SPOOFING               | HIGH         | PTP grandmaster spoofing (port 319 UDP)             |
| OT_PLC_MODE_SWITCH            | CRITICAL     | Rapid PLC RUN/STOP mode switching                   |
| OT_WATCHDOG_MANIPULATION      | CRITICAL     | Watchdog timer disabled or manipulated              |
| OT_VALVE_CYCLING              | CRITICAL     | Rapid valve open/close commands                     |
| OT_MOTOR_CYCLING              | CRITICAL     | Rapid motor start/stop commands                     |
| OT_UNAUTHORIZED_MODBUS_ACCESS | HIGH         | Modbus access from unknown host                     |
| OT_PROTOCOL_ANOMALY           | MEDIUM       | Generic OT protocol anomaly (Zeek notice)           |

## 13.2 Key File Paths

| Item                      | Path                                                           |
|---------------------------|----------------------------------------------------------------|
| Main config               | /opt/SmartShield-Factory/config/smartshield.yaml               |
| OT protected devices      | /opt/SmartShield-Factory/config/ot_protected_devices.conf      |
| Authorized Modbus masters | /opt/SmartShield-Factory/config/modbus_authorized_masters.conf |
| Redis config              | /opt/SmartShield-Factory/config/redis.conf                     |
| Whitelist                 | /opt/SmartShield-Factory/config/whitelist.conf                 |
| TI feeds list             | /opt/SmartShield-Factory/config/TI_feeds.csv                   |
| Service environment file  | /etc/smartshield/environment                                   |
| Systemd service           | /etc/systemd/system/smartshield.service                        |
| Redis data                | /var/lib/smartshield/redis/                                    |
| Alerts output             | /var/lib/smartshield/output/alerts.json                        |
| Blocking log              | /var/lib/smartshield/output/blocking.log                       |
| Zeek logs                 | /var/lib/smartshield/output/zeek/                              |
| System logs               | journalctl -u smartshield                                      |

## 13.3 Port Reference

| Port          | Protocol           | OT Devices                                                |
|---------------|--------------------|-----------------------------------------------------------|
| 502/TCP       | Modbus/TCP         | PLCs, remote I/O modules, Modbus gateways                 |
| 102/TCP       | S7Comm (ISO-TSAP)  | Siemens S7-300, S7-400, S7-1200, S7-1500 PLCs             |
| 20000/TCP+UDP | DNP3               | RTUs, substations, water/power utility systems            |
| 4840/TCP      | OPC-UA             | SCADA servers, historians, modern PLC gateways            |
| 34964/UDP     | PROFINET DCP       | Siemens PROFINET devices, distributed I/O                 |
| 44818/TCP     | EtherNet/IP CIP    | Allen-Bradley (Rockwell) PLCs, ControlLogix, CompactLogix |
| 123/UDP       | NTP                | Time synchronization for PLCs and SCADA servers           |
| 319/UDP       | PTP (IEEE 1588)    | Precision time for motion control and substations         |
| 55000/TCP     | SmartShield Web UI | Management interface — restrict to management VLAN only   |

| Port     | Protocol | OT Devices                                                |
|----------|----------|-----------------------------------------------------------|
| 6379/TCP | Redis    | Internal only — localhost binding, not exposed externally |